Task 1 — Web App QA & Debug Report

RealWorld / Conduit React application

Tester: Vishal Kumar Yadav
Test Environment: Windows 11, Microsoft Edge, local application at http://localhost:3000
Test Date: 9 November 2026
Scope: Sign-up, login, content/editor access, navigation, API failure handling, validation, and UI/resource loading.

1. Executive Summary

The application was tested locally through the main user flows requested in the assessment. The home page repeatedly attempted to load article and tag data but remained on a Loading state while the API requests returned HTTP 530. The same API communication problem prevented successful login and registration. The sign-up form also allowed submission with all fields empty instead of presenting clear required-field validation. The main stylesheet returned HTTP 404 on the editor/login flow, causing the interface to render as largely unstyled HTML. The editor route redirected unauthenticated users to the login page, which was treated as expected access control rather than a defect. A clean browser-console refresh did not reveal an additional independent JavaScript exception beyond the observed network/API failures.

2. Test Observations

Area	Observed Result
Home	Page remained at “Loading...” while article/tag requests repeatedly returned HTTP 530.
Login	Authentication requests failed with HTTP 530/CORS-related errors; no useful user-facing failure message was observed.
Sign-up	Empty fields could be submitted and a users request was generated; no clear required-field messages were shown.
Editor	Direct access while unauthenticated redirected to /login; treated as expected behavior.
Styling	main.css returned HTTP 404, leaving the application visibly unstyled.
Console	After a clean refresh, no independent red JavaScript exception was observed in the console screenshot.

3. Defect Log

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Home page remains stuck in Loading state when API calls fail	Open / and wait while article/tag data loads. Inspect Network → Fetch/XHR.	**Expected:** The UI should transition to a clear error state and/or provide a retry action when the data service is unavailable. **Actual:** The page remains on “Loading...” while article/tag requests fail with HTTP 530.	High	Frontend loading state does not visibly transition to an actionable error state when the data requests fail.
2	Excessive repeated API requests after API failure	Open /, select Network → Fetch/XHR, and wait without further interaction.	**Expected:** Failed requests should be bounded or user-controlled after an error. **Actual:** Repeated articles and tags requests continue automatically; multiple requests return 530 and some remain pending.	Medium	An automatic retry/refetch mechanism appears to continue without an effective stopping condition or user-controlled retry flow.
3	Login failure provides no useful user-facing error	Open /login, enter test credentials, submit, and inspect the authentication request.	**Expected:** Successful credentials should authenticate, while an unavailable/failed service should produce a clear error message. **Actual:** Login requests fail with HTTP 530/CORS-related errors and authentication does not complete; no useful user-facing error is shown.	High	The authentication request cannot complete against the configured API, and the frontend does not expose a clear failure state to the user.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	Sign-up form submits with empty required fields	Open /register, leave name, email, and password empty, then click Sign up.	**Expected:** Required-field validation should block submission and identify missing fields; no API request should be made. **Actual:** The form submits and a users request is generated; no clear required-field validation is displayed.	Medium	Missing client-side required-field validation before the registration request.
5	Main stylesheet returns HTTP 404	Open the application/editor flow and inspect Network for CSS resources.	**Expected:** The main stylesheet should load successfully and the application should render with its intended styling. **Actual:** main.css returns HTTP 404 and the UI renders largely as unstyled HTML.	Medium	Incorrect/missing CSS asset path or missing stylesheet in the local build/static configuration.

4. Root Cause Analysis — Issue #1

The home page depends on remote article and tag API requests to populate its main content.

During testing, both requests repeatedly returned HTTP 530 and the browser console also showed CORS-related failures for the same API resources.

Because the required data could not be retrieved, the application did not receive the information needed to leave its initial loading state.

The visible UI remained at “Loading...” instead of presenting an error state that explained the service failure.

The Network panel showed repeated requests being generated without additional user interaction, indicating an automatic retry/refetch behavior.

This repeated activity increases unnecessary network traffic while still leaving the user without usable content.

A more resilient implementation would distinguish loading, success, and error states and stop automatic retries after a bounded number of attempts or make retry user-controlled.

The exact upstream reason for the HTTP 530 response was not changed or repaired during this QA exercise; the report therefore treats the observed API/CORS failure and frontend error-state handling as the evidence-based scope of the defect.

5. Evidence Screenshots

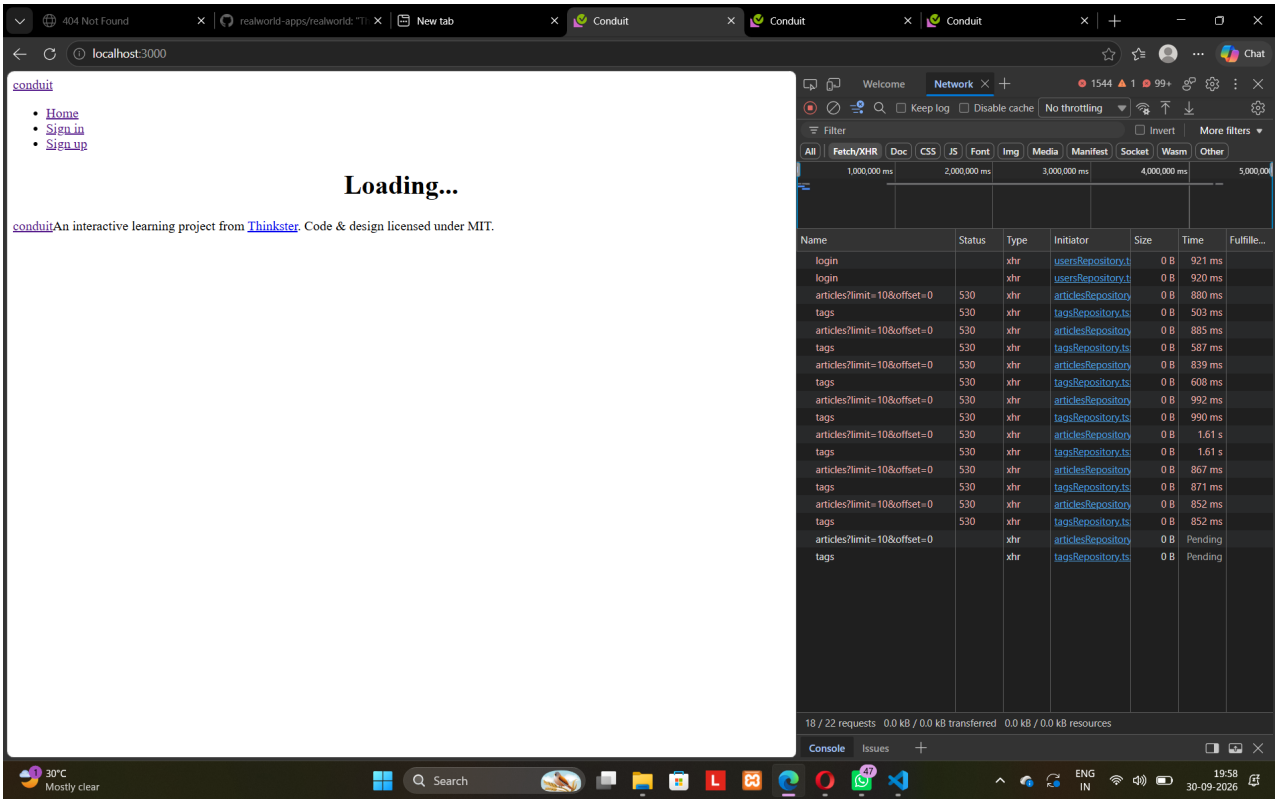


Figure 1 — Home page stuck on Loading with repeated article/tag HTTP 530 requests (evidence for Issues #1 and #2).

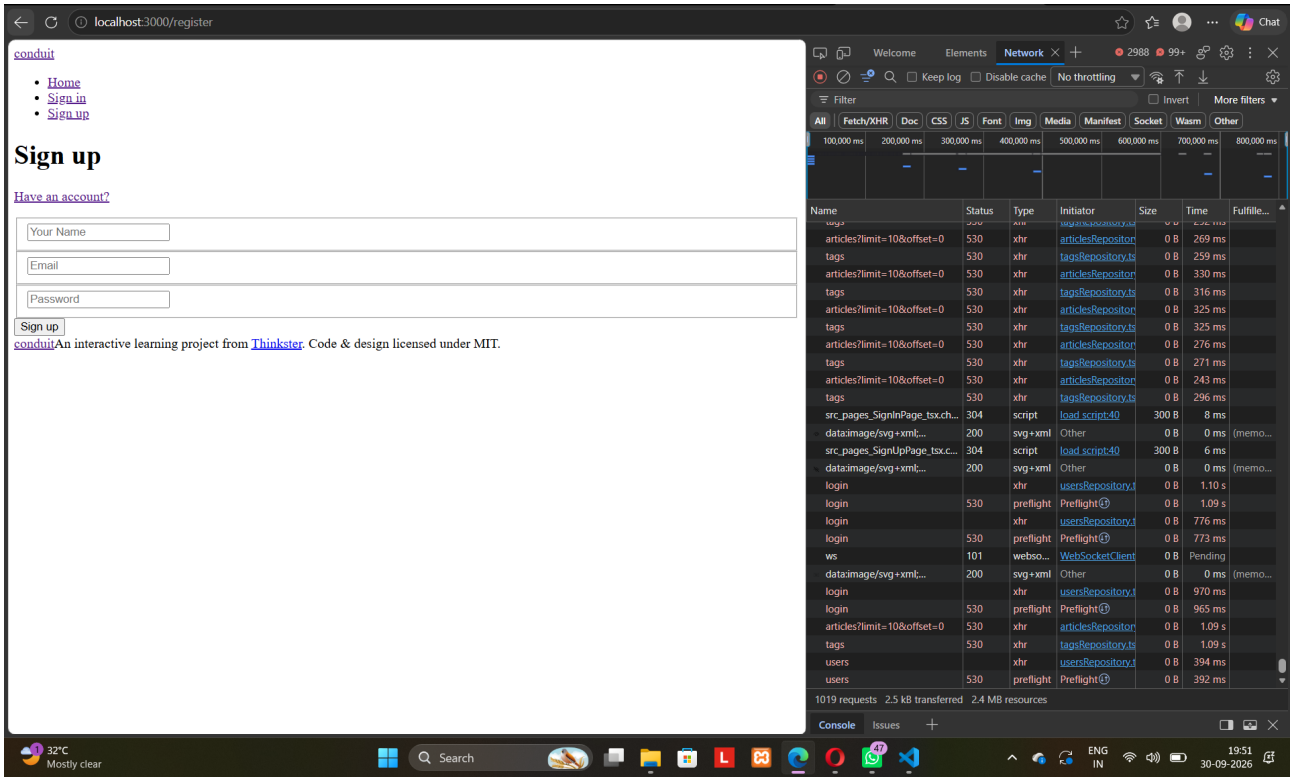


Figure 2 — Empty sign-up form with users request activity visible in Network (evidence for Issue #4).

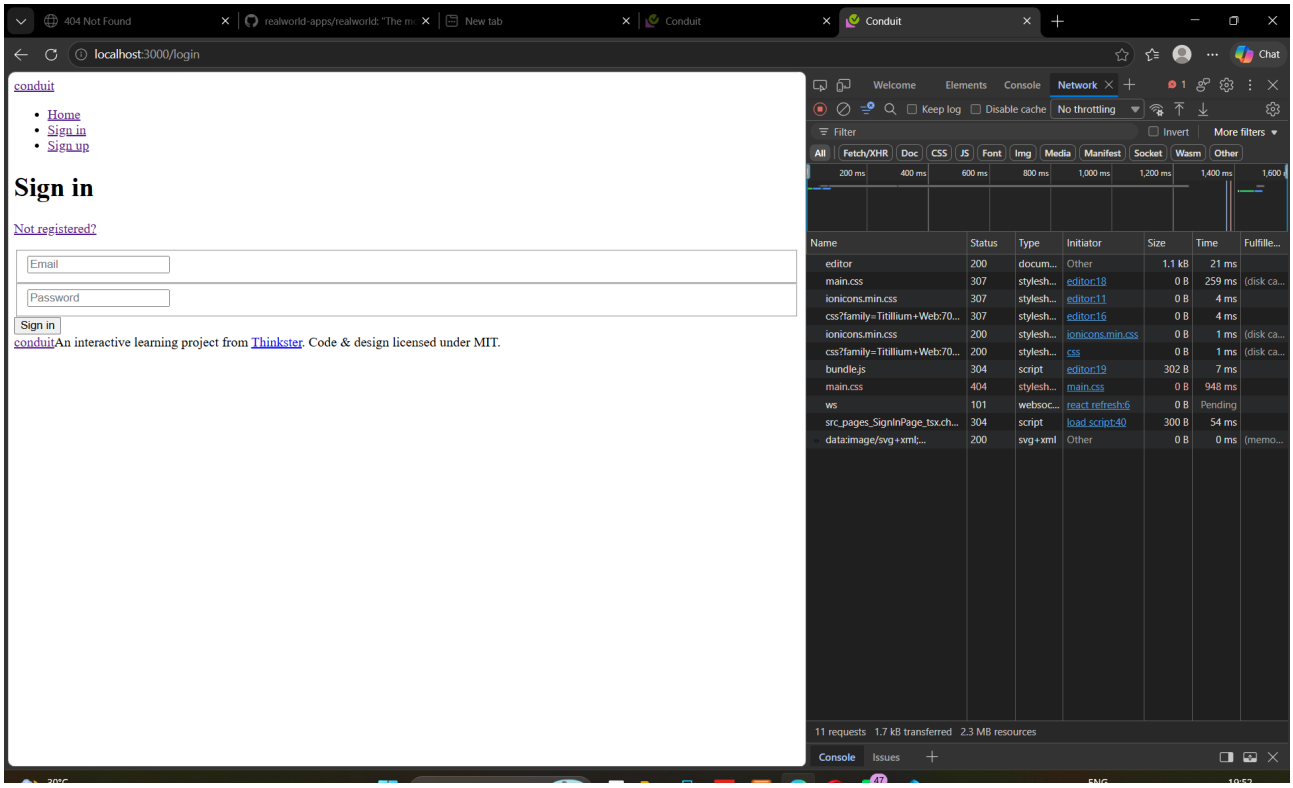


Figure 3 — Login/editor flow showing main.css returning HTTP 404 (evidence for Issue #5).

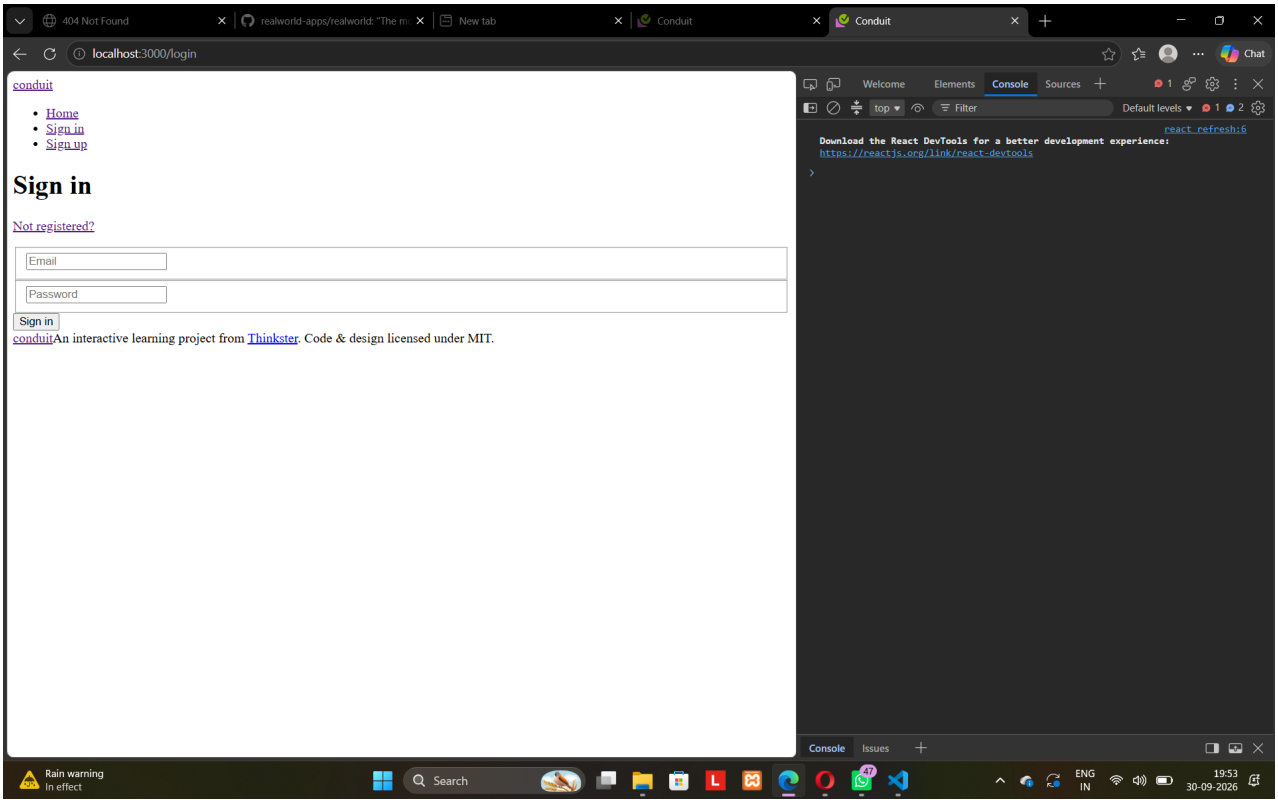


Figure 4 — Clean Console after refresh; React DevTools notice is informational and not treated as a defect.

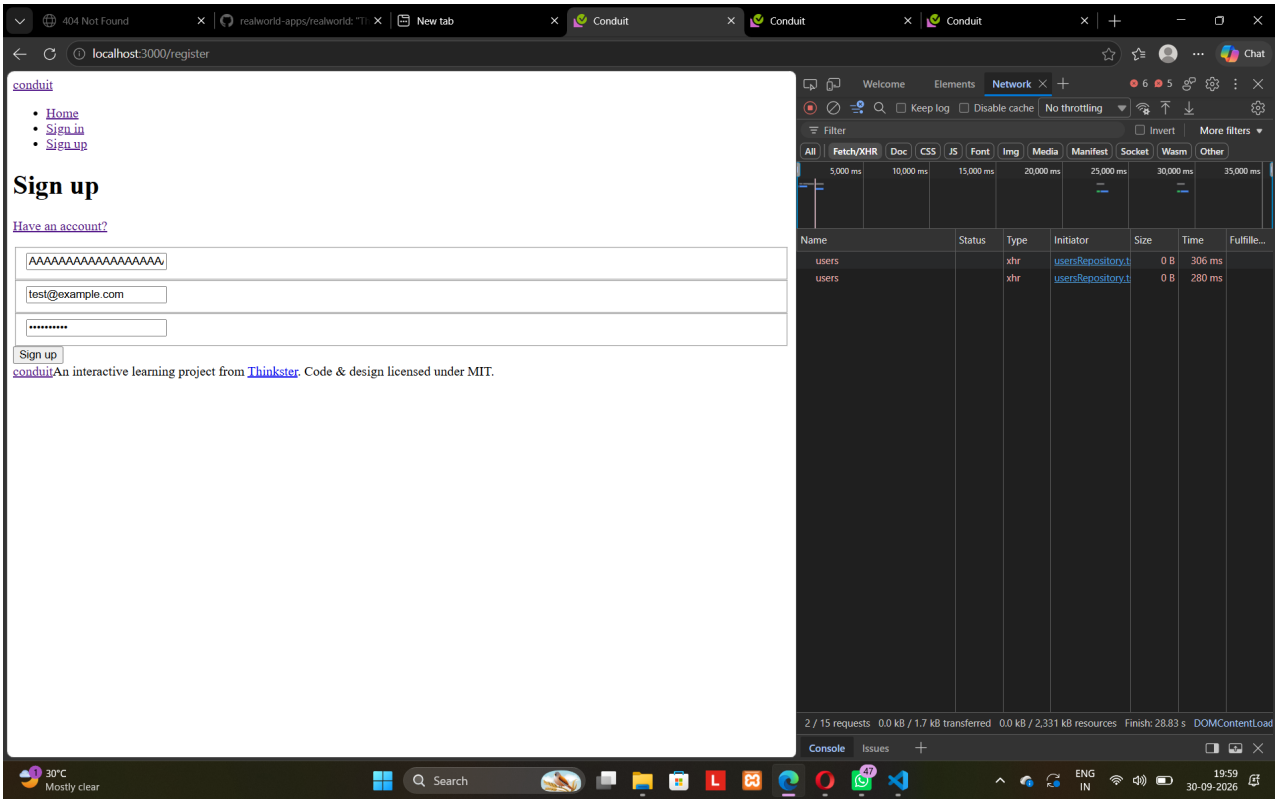


Figure 5 — Long username test; form accepted the input and generated users requests. This was not counted as a separate defect because the backend/API failure prevents attributing the outcome to input-length handling.

6. Limitations

Successful authentication could not be completed because the configured API requests were failing during the test session. Consequently, authenticated create/edit/delete article operations could not be executed end-to-end. Direct unauthenticated access to /editor redirected to /login, which was considered expected access control. The report records only behavior observed during the local test session and does not claim that the upstream API itself was permanently unavailable.

7. Conclusion

The main QA risks observed were API failure handling, repeated failed network activity, authentication error feedback, missing form validation, and a broken stylesheet resource. The findings are reproducible from the provided screenshots and browser observations and are suitable for the requested Task 1 QA/debug report.